Faculdade de Engenharia da Universidade do Porto

PSD – Projecto de Sistemas Digitais

Implementação de uma unidade de cálculo parametrizável para vírgula flutuante

Realizado por:

Pedro Luís Pereira Portela dos Reis – ee00106
Rui Teodoro Barreto Barros – ee00042

Professor **António José Duarte Araújo**

INTRODUÇÃO

Este trabalho consiste numa unidade de cálculo em vírgula flutuante parametrizável que executa somas, subtracções, multiplicações e divisões.

Usamos, conforme a norma IEEE para vírgula flutuante (IEEE 754), números denormalizados o que complicou os circuitos (e atrasou) comparativamente com o uso de apenas normalizados.

A divisão estava a ser implementada para suportar igualmente números denormalizados, porém devido a escassez de tempo, optou-se por não o fazer. Caso se pretenda implementar esta funcionalidade terá de se adicionar o bloco que põe o hidden bit, alterar o bloco divisor (ou colocar antes deste um bloco para retirar os zeros mais significativos da mantissa, reportando) e alterar o bloco normalizador da divisão.

Existem duas implementações de divisão e multiplicação: uma combinacional e outra sequencial, e pode-se optar por usar uma ou outra nos módulos gerais destas duas operações, comentando a evocação de um ou outro bloco.

Os resultados poderiam ser melhorados (em termos de área e tempo de execução) caso fossem retirados os bits de guarda e criação do sticky bit, mas optou-se por deixar aberto a implementações futuras.

Os operandos de entrada e saída têm um determinado número de bits, o mais significativo para sinal, os seguintes para expoente e os menos significativos para a mantissa.

Para os testes foram usados 8, 16 e 32 bits. Para 8 bits usou-se 3 bits para o expoente e 4 bits para a mantissa. Para 16 bits usou-se 6 bits para o expoente e 9 bits para a mantissa. Para 32 bits usou-se o formato normalizado de 8 bits para o expoente e 23 bits para a mantissa.

Está implícito que existe um hidden bit igual a 1 quando o expoente é diferente de 0, e igual a 0 quando o expoente é 0. Este formato “denormal” permite dar um maior alcance ao expoente, garantindo muito mais casas binárias (mas com menor precisão quando o hidden bit é 0, pois, tem menos casas binárias).

Como saídas do circuito incluiram-se as flags overflow, underflow, divisão por zero e indeterminação. Overflow corresponde ao caso em que, executando uma operação com números não especiais, obtemos um valor acima do valor máximo definido para o número de bits em questão, ou seja, um valor com o expoente igual ou superior a tudo 1s. Underflow ocorre quando, executando uma operação com números não especiais, obtemos um valor que, ficando o expoente a 0 (hidden bit a 0), a sua mantissa fica também a 0. Indeterminação ocorre quando numa operação temos uma operação entre dois operandos não NaN cujo resultado fica NaN.

Não se considera indeterminação quando um dos operandos de entrada é NaN (e consequentemente a saída é NaN) pois não se trata de uma indeterminação, mas da passagem de um operando para a saída; também não se considera a ocorrência de um infinito na saída um overflow pois a informação que se quer dar ao utilizador quando ocorre um overflow é que não houve alcance suficiente da unidade VFL para efectuar a operação entre os operandos na entrada, e isso não tem relação com o tratamento de casos especiais cujo resultado é infinito.

Sendo uma unidade parametrizável, para se fazer a alteração do formato de virgula flutuante a usar, basta alterar o ficheiro “parameters.vh”. Estão de momento lá presentes, exemplos de formatos de 8, 16 e 32 bits; sendo este último correspondente ao formato simples.

DESCRIÇÃO DOS CIRCUITOS

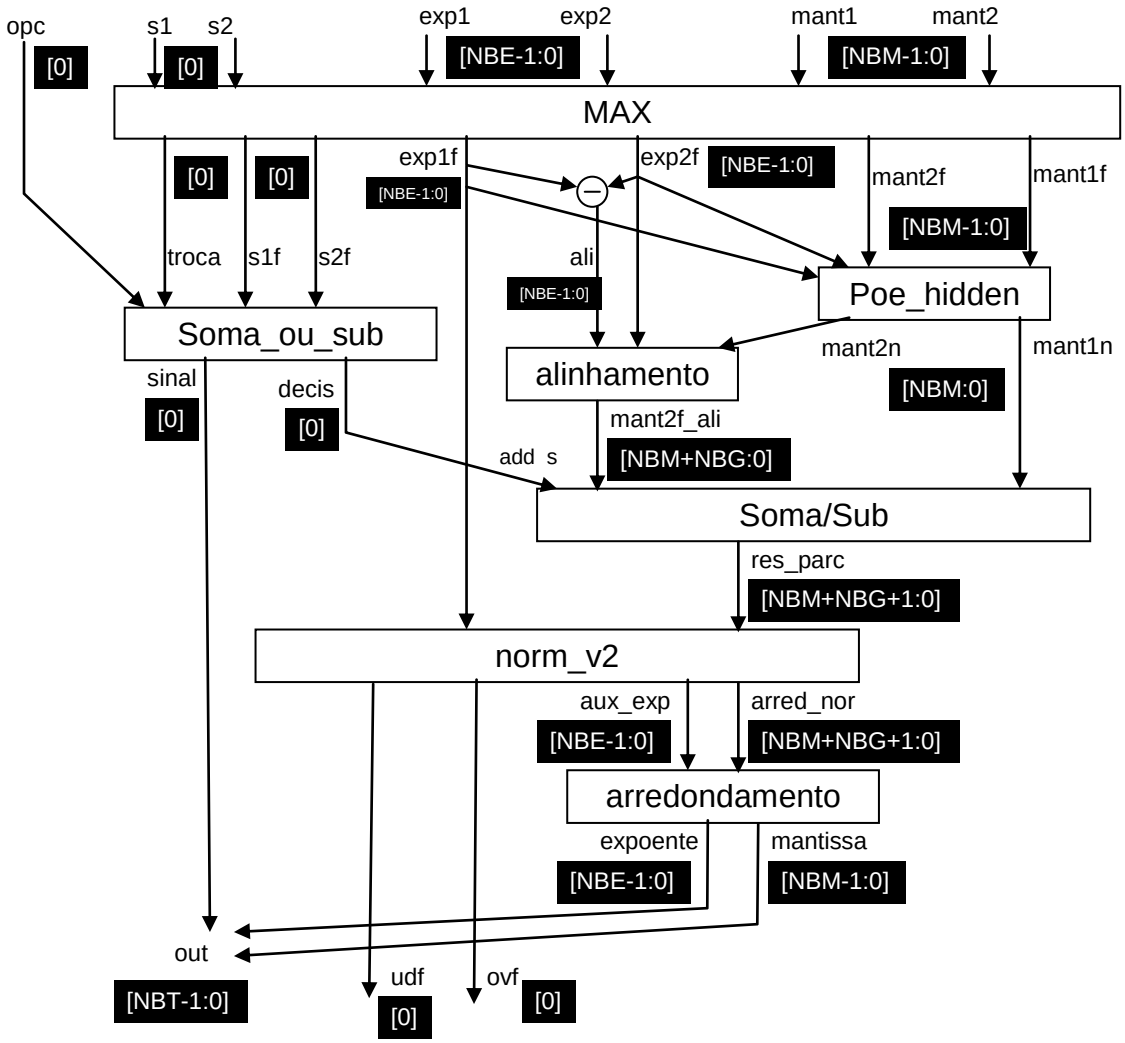
O nosso módulo principal (VFLunit) tem como entradas o opcode (dois bits que caracterizam a operação a realizar → 00 para a soma, 01 para a subtracção, 10 para a multiplicação e 11 para a divisâo), os operandos de entrada (com o bit de sinal, os bits de expoente e os bits da mantissa), o clock, reset e start (para controlarem a parte sequencial do circuito). Como saídas tem o operando com o resultado da operação, um conjunto de 4 flags (indterminação, divisão por zero, overflow e underflow) e uma flag adicional que indica que a operação foi realizada e o operando na saída é o resultado (done).

Quatro módulos correm em paralelo: val_exp_geral (que trata dos casos especiais), somasub_geral (que trata da soma e da subtracção), multiplicacao_geral (que trata da multpilicação) e divisao_geral (que trata da divisão). O módulo arred (que trata do arredondamento) está sempre a correr mas só é necessário quando o operando final de algum dos módulos anteriores é calculado. A saída de um dos módulos é escolhida baseando-se nos bits opcode da entrada e na flag devolvida pelo módulo dos valores especiais que tem precedência sobre o opcode.

Começamos por fazer uma descrição geral do somador/subtrator, mostrando o esquema geral de blocos usado, e é dada uma explicação detalhada dos vários blocos. Conclui-se com uma descrição semelhante para a multiplicação, divisão, bloco dos valores especiais e arredondamento.

Secção 1.01 Somador

Esquema geral de blocos:



Nota: Nas caixas a preto são apresentados os tamanhos dos sinais.

Descrição Geral do Somador:

O circuito somador e subtrator estão integrados num único pois a operação de soma ou subtração é traduzida por um único bloco bastante optimizado que recebe um sinal a indicar a operação pretendida. Assim, também é poupado bastante espaço da FPGA.

Inicialmente são ordenados os números por valor absoluto, de modo ao primeiro ficar maior que o segundo. Isto irá ter utilidade para a subtracção dos expoentes e alinhamento da mantissa do menor dos números, bem como para o bloco "Soma_ou_sub" como à frente será explicado.

Posteriormente é adicionado o hidden bit (bit mais significativo) nas mantissas de acordo com o expoente dos números: caso seja zero é colocado a zero (*número denormalizado*), senão é colocado um "1" (*número normalizado*).

A mantissa do segundo número (o menor em módulo) é seguidamente alinhada de acordo com a diferença entre os expoentes, com a excepção do expoente de 2 (segundo número) ser zero (ver adiante na descrição do bloco).

Para a decisão da operação a realizar (soma ou subtracção) que será indicada ao bloco "Somasub" temos o bloco "Soma_ou_sub" que tem também a tarefa de calcular o valor do sinal do resultado. A decisão da operação é explicada mais à frente. O sinal poderia ser calculado antes do bloco "Max", mas optou-se por calcular em paralelo com os blocos de subtracção de expoentes, pôr o hidden bit e alinhamento (como podemos ver mais à frente nos resultados só o bloco de alinhamento demora mais tempo a executar que este).

Após a realização da operação propriamente dita, teremos de normalizar o número ajustando a mantissa e o expoente. Como existem excepções (utilização de números denormalizados) e a normalização da mantissa é dependente do expoente, bem como vice-versa optou-se por fazer um só módulo com os sinais de entrada: expoente e mantissa não normalizados.

O último módulo faz o arredondamento e só está implementado o método de truncatura, podendo-se de futuro implementar outros métodos (o expoente já consta como entrada para eventual modificação em caso de arredondamento) e eventualmente colocar um sinal de entrada do circuito geral que seleccione entre os métodos de arredondamento. É neste último módulo que se retirada o hidden bit e os bits de guarda.

Bloco "Max":

Este bloco tem a tarefa de trocar os números de entrada no caso do segundo ser superior, em módulo, que o primeiro – assinalando a flag "troca" – ou de manter a ordem destes em caso contrário ou de igualdade.

Assim, é inicializada a flag a zero, vê-se se os expoentes são iguais: caso afirmativo, são comparadas as mantissas e executa-se a operação respectiva; caso negativo, são comparados os expoentes e é executada a operação. Em caso de operação de troca altera-se a flag.

Bloco "som_ou_sub":

Este bloco tem a tarefa de determinar a operação a realizar por "Somasub", se soma ou subtracção, bem como determinar o sinal do resultado final.

A lógica usada neste bloco pode ser explicada pela seguinte tabela:

Troca	Sinal de 1	Sinal de 2	Opcode	Operação	Sinal final
0	0	0	0	0	S1 = 0
0	0	0	1	1	S1 = 0
0	0	1	0	1	S1 = 0
0	0	1	1	0	S1 = 0
0	1	0	0	1	S1 = 1
0	1	0	1	0	S1 = 1
0	1	1	0	0	S1 = 1
0	1	1	1	1	S1 = 1
1	0	0	0	0	0
1	0	0	1	1	1
1	0	1	0	1	0
1	0	1	1	0	1
1	1	0	0	1	1
1	1	0	1	0	0
1	1	1	0	0	1
1	1	1	1	1	0

TABELA 1: Decisão da operação a realizar no bloco somador e do sinal do resultado.

De notar que para "Troca"=0 (flag proveniente do bloco "max") o resultado da decisão de "Operação" é idêntico a "Troca"=1 – Xor (entre S1 e S2) para "Opcode"=0 ou /Xor se "Opcode"=1 – e que o sinal do resultado final, caso "Troca" seja 0, é sempre S1.

Seguiu-se a realização do mapa de Carnot para "Troca"=1 de forma a descobrir o sinal final e, juntando tudo, obteve-se o seguinte circuito lógico:

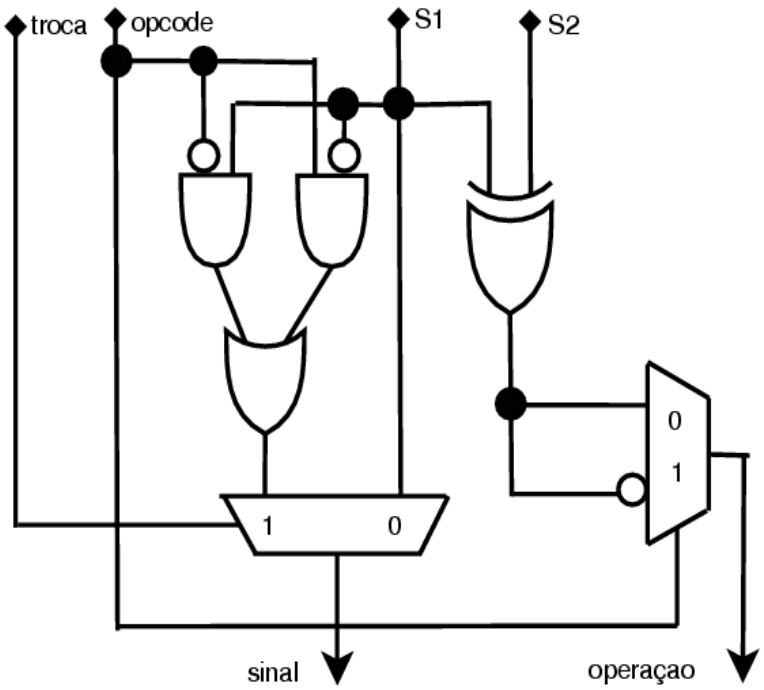


Figura 1: Circuito lógico para decisão do sinal final e operação a realizar

Bloco “põe_hidden”:

Aqui é adicionado mais um bit às mantissas (ou significandos), o hidden bit. Para cada número, vê-se o expoente: caso seja zero o hidden bit fica zero; caso seja outro, o hidden bit fica um. O resto da mantissa é mantido igual.

Bloco “alinhamento”:

Para se poder à operação de soma ou subtracção, a segunda mantissa terá de ser alinhada pela diferença entre os expoentes dos dois números (“ali”). Para o caso normal, o alinhamento é um simples deslocar da mantissa de “ali” posições para a direita; no caso do expoente ser zero (número denormalizado) são deslocadas “ali”-1 posições já que o expoente a zero representa o mesmo expoente que um. De notar, para este último caso, que é necessário indicar que a mantissa fica igual no caso de “ali” ser zero, porque senão estaríamos a deslocar “-1” posições para a direita.

Optou-se por colocar logo aqui os bits de guarda que serão usados na soma/subtracção.

Bloco “somasub”:

É neste bloco que é feita a operação de soma ou de subtracção propriamente dita. Antes da operação, decidida por “add_sub” (proveniente de “som_ou_sub”), são adicionados bits a zero na posição antes do hidden bit. Isto servirá para o resultado (“res_parc”) apresentar um bit para o carry da soma, 5 bits para o número e NBG (parâmetro estabelecido com 2) para os bits de guarda. Aqui são apenas colocados os bits de guarda (a zero) no primeiro número, já que no segundo já tinham sido postos no bloco de alinhamento.

Bloco “norm_v2”:

Este é o bloco da normalização. Vai percorrendo os bits do mais significativo para o menos significativo, quando o resultado não é denormal, à procura do primeiro 1 ou até o expoente (que em cada iteracção vai decrementando 1) ser 0, ou seja, o número passar a denormal. Mal passe a denormal, já não é preciso shiftar ou decrementar o expoente, logo já não entra no if (linha 30). No caso em que temos no final o hidden bit a 1 e o auxexp a 0 temos de colocar o auxexp a 1, pois o hidden bit representa uma casa binária a mais que vai ser compensada incrementando o expoente (pondo-o a 1).

Ainda neste bloco é determinado se há overflow ou underflow. Considerando números denormal este é o único bloco onde tal é possível pois tem de se ter em atenção o expoente e mantissa finais (após normalização). Assim, se todos os bits do expoente forem 1 então temos overflow “*if(&auxexp)*” e a mantissa (arred_nor) fica a 0. Caso não seja overflow, então a flag ovf fica a 0.

Se for underflow, situação que acontece se o expoente (auxexp) e a mantissa (arred_nor) forem 0, coloca-se a flag udf a 1. Se não, udf fica a 0.

Secção 1.02 Valores especiais

Bloco "val_exp_geral":

Dividiu-se a tarefa de detectar valores especiais, calcular o seu resultado e passá-lo para a saída, em três módulos: val_exp, val_exp2_v2 e val_exp3. Neste bloco determina-se se alguma das variáveis é mais ou menos infinito, zero ou Not a Number e, caso sim, de acordo com a operação a realizar, é lançado para a saída o resultado. No caso de soma ou subtracção de uma constante com zero, o resultado é calculado fora do bloco dos valores especiais.

Bloco "val_exp":

O primeiro módulo (val_esp.v), através dos valores passados (sinal, expoente e mantissa dos dois operandos) detecta se cada um desses é algum valor especial.

Primeiramente, este módulo identifica se os expoentes têm tudo a 1 (exp1_ff para o primeiro operando, exp2_ff para o segundo operando) ou tudo a zero (exp1_zero para o primeiro operando, exp2_zero para o segundo) e se as mantissas têm tudo a 0 (inf1 para o primeiro operando e inf2 para o segundo operando), pois são estas as características dos valores especiais. "*&exp1*" retorna 1 se exp1 for tudo 1s, "*!(|exp1)*" retorna 1 se exp1 for tudo 0s e "*!(|mant1)*" retorna 1 se mant1 for tudo 0s.

Seguidamente, é testado através das características obtidas se os operandos são valores especiais. Pelo quadro abaixo pode-se ver que um operando é zero se expoente e mantissa tiverem apenas 0s, independentemente do valor do sinal: assim o operando 1 é zero se a intersecção de exp1_zero e inf1 for verdadeira (igual a 1). Para o operando ser mais infinito $+\infty$, o sinal tem de ser 0, o expoente com tudo a 1s e a mantissa com tudo a 0s (intersecção de exp1_ff, inf1 e !s1 para o primeiro operando). Para o operando ser $-\infty$, o sinal tem de ser 1, o expoente com tudo a 1s e a mantissa com tudo a 0s (intersecção de exp1_ff, inf1 e s1 para o primeiro operando). Para que o operando seja um Not a Number (NaN), o expoente tem de ser tudo 1s e a mantissa não poderá ser 0 (intersecção de exp1_ff e !inf1 para o primeiro operando). O operando será um valor não especial (cte_1 para o primeiro operando, cte_2 para o segundo) quando não for qualquer um dos casos anteriores.

Operando	Sinal	Expoente	Mantissa
0	x (= 0 v 1)	00…0	00…0
$+\infty$	0	11…1	00…0
$-\infty$	1	11…1	00…0
NaN	x	11…1	xx…x ($\neq 0$)

Bloco "val_exp2_v2":

As entradas do segundo módulo (val_exp2_v2.v) são as saídas do primeiro mais o sinal dos dois operandos e a operação (soma, subtracção, multiplicação ou divisão) a realizar, transmitidas através da variável op[1:0]. O objectivo é definir quando o

resultado da operação entre dois operandos tem como resultado zero, NaN, mais infinito ou menos infinito (quando obviamente um dos operandos é um valor especial). Este é o módulo mais complicado de implementar e sujeito a muitos erros fortuitos. Na busca por poupar tempo de execução e tamanho do circuito, definiu-se cada um dos casos separadamente juntando todas as possibilidades ligadas a estes casos através de portas lógicas OR, NAND e duas XOR. O quadro abaixo serviu de referência, tentando-se sempre que possível agrupar casos diferentes de modo a poupar portas lógicas. Por exemplo, facilmente se vê que quando um dos operandos é NaN, então o resultado da operação será NaN (isto é facilmente agrupável → "*nan_1* | *nan_2*"). Foi adoptada uma política de primeiramente separar os casos por tipos de operações (adição, …) e, em cada um destes tipos, procurar agrupar os NaN, +∞, -∞ ou 0.

Toda a compreensão das quatro primeiras linhas "assign" reside nestes pressupostos e nas tabelas abaixo:

Adição (x + y)					
x	y				
	0	NaN	+∞	-∞	constante
0	0	NaN	+∞	-∞	constante
NaN	NaN	NaN	NaN	NaN	NaN
+∞	+∞	NaN	+∞	NaN	+∞
-∞	-∞	NaN	NaN	-∞	-∞
constante	constante	NaN	+∞	-∞	constante

Subtracção (x - y)					
x	y				
	0	NaN	+∞	-∞	constante
0	0	NaN	-∞	+∞	constante
NaN	NaN	NaN	NaN	NaN	NaN
+∞	+∞	NaN	NaN	+∞	+∞
-∞	-∞	NaN	-∞	NaN	-∞
constante	constante	NaN	-∞	+∞	constante

Multiplicação (x * y)					
x	y				
	0	NaN	+∞	-∞	constante
0	0	NaN	NaN	NaN	0
NaN	NaN	NaN	NaN	NaN	NaN
+∞	NaN	NaN	+∞	-∞	±∞
-∞	NaN	NaN	-∞	+∞	±∞
constante	0	NaN	±∞	±∞	constante

Divisão (x / y)					
x	y				
	0	NaN	+∞	-∞	constante
0	NaN	NaN	0	0	0
NaN	NaN	NaN	NaN	NaN	NaN
+∞	+∞	NaN	NaN	NaN	±∞
-∞	-∞	NaN	NaN	NaN	±∞
constante	±∞	NaN	0	0	constante

flags[2] e flags[3] representam, respectivamente, divisão por zero e indeterminação. A divisão por zero ocorre na divisão "*(op==2'b11)*" e quando o segundo operando é zero e o primeiro não é (caso o primeiro e o segundo operandos fossem zero, tratar-se-ia de uma indeterminação). Uma indeterminação ocorre para os casos ∞ + (-∞), +∞ - (+∞), -∞ + (+∞), -∞ - (-∞), +∞ * 0, -∞ * 0, 0 * +∞, 0 * -∞, 0 / 0, +∞ / +∞, +∞ / (-∞), -∞ / (-∞) e -∞ / +∞.

Bloco "val_exp3":

O terceiro módulo (val_exp3.v), tendo como entradas os bits NaN, zero, mais_inf e menos_inf que dizem se o operando final vai ser Not a Number, zero, mais infinito ou menos infinito, respectivamente, criando saídas que representam estes valores especiais. Estas saídas codificam-se segundo o tabela dos valores especiais atrás definida. Note-se no entanto que a mantissa do Not a Number foi definida como 00…1, uma vez que podia ser qualquer valor excepto zero, e, desta maneira, só se altera um bit (pois já tinha sido inicializada com 0).

Secção 1.03 Multiplicação

Em termos muito gerais, a multiplicação divide-se em três módulos diferentes que se interligam (o módulo que põe o hidden bit nos dois operandos – poe_hidden_m – o módulo que realiza a operação de multiplicação propriamente dita – multiplicacao – e o módulo de normalização – norm_mul_v2) e algumas operações adicionais.

Primeiramente determina-se o expoente final caso não houvesse carry na multiplicação ou operandos denormal: exp_t1, que é a soma dos expoentes dos operandos menos o BIAS: a subtracção do BIAS dá-nos logo uma informação sobre a situação virtual do operando (virtual pois ela pode mudar na normalização): se há overflow ou underflow. Se houver overflow então exp1_t1 será 011…1 ou 10xx…x, se houver underflow então exp1_t1 será 11xx…x ou 0 com mantissa tb a 0 (caso dos denormal): mas estas situações só são calculadas numa primeira fase após a normalização e numa segunda fase, depois do arredondamento. Na primeira fase, como pode haver deslocamento de um bit na normalização, também se tem de considerar a hipótese de não haver overflow com exp_t1 igual a tudo 1s; devido à maneira como foi tratada a normalização há a hipótese de não ser lá detectado um underflow pelo que no final se considerou que o underflow da multiplicação (ovfm) era o OR da flag underflow da normalização (udf2) com o underflow calculado no bloco principal (udf). O overflow da multiplicação (ovfm) é detectado no bloco da normalzação.

Em segundo, é colocado o hidden bit nos dois operandos: bloco poe_hidden_m. Depois é realizada a operação de multiplicação propriamente dita: bloco multiplicacao para multiplicação combinacional ou bloco multiplicacaoseq para multiplicação sequencial. Para passar de um bloco para outro basta, no caso da multiplicação combinacional comentar da linha 77 à 88 inclusivé, e no caso da multplicação sequencial, comentar da linha 65 à 74 inclusivé.

Em seguida faz-se a normalização (bloco norm_mul_v2), define-se a mantissa de saída (arred_nor2), para que só nesta altura ela mude (para não haver salto de bits ao

longo da operação, que pode provocar atrasos e indefinição na entrada. E finalmente são definidas as flags overflow e underflow, que foram esplicadas anteriormente.

Bloco poe_hidden_m:

Este bloco coloca o hidden bit a 1 quando o expoente é diferente de 0 pois, caso contrário, temos um operando denormal cujo hidden bit é 0. Assim temos na saída o operando com mais um bit que o que tinha na entrada, na posição mais significativa.

Bloco multiplicacao:

Neste bloco é realizada a operação de multiplicação dos dois operandos com os hidden bits incluídos. O resultado vai ficar em mul_norm_temp que tem de ficar com 2*NBM+2 bits, que corresponde ao resultado da multiplicação (com um possível carry). É definido também o sticky bit que indica se há algum 1 na zona da mantissa fora dos bits que vão ser "pescados" para o próximo módulo (hidden bit + NBM da mantissa + NBG de guarda) e que serviria para o bloco de arredondamento (não implementado). Finalmente, mul_norm guarda os bits do resultado da multilicação desde o mais significativo até 1+NBM+NBG bits a partir do mais significativo que incluem o hidden bit, os bits reservados para a mantissa e os bits de guarda.

O facto de nos assign, as posições dos bits estarem definidas como subtracções redundantes quando podiam ser definidas mais directamente explica-se na melhor visualização do que se pretende por parte de quem vê o código, sem maiores encargos para os circuitos da FPGA visto que o compilador automaticamente efectua esses cálculos.

Bloco multiplicacaoseq:

Na multiplicação sequencial foi usado um multiplicador sequencial shift-add. Todo o processo sequencial se baseia numa transição positiva de clock (enquanto o reset se encontra a 1, ou seja, quando o circuito está ON).

state a 0 significa que estamos na prmeira iteração e é aqui que, caso start esteja a 1 (operandos carregados e prontos a serem processados), multreg é carregado nos NBM+1 bits menos significativos com o multiplicador (mantendo-se os outros bits a 0) e a contagem dos ciclos começa ("*cntbits <= 0*"). state é passado para 1 para que na iteração seguinte o processo passe para os procedimentos seguintes. Quando state é 1, inicia-se o processo multiplicativo shift-add. Primeiramente, testa se já realizou todos os ciclos, e se sim, coloca done a 1(representando que o resultado está calculado e pode ser retirado) e state volta a 1, ficando o circuito à espera do próximo start. Caso não, então investiga o primeiro bit do multiplicador: se este estiver a 0, não é preciso somar nada a multreg (que fica com o resultado da multiplicação a paritr da posição mais significativa), se estiver a 1, então é somado a multreg o multiplicando. Por precaução, state é colocado a 1 e done a 0 (pois o resultado ainda não está calculado).

O produto fica com o resultado da multiplicação, que são os seguintes bits: hidden bit, bits da mantissa normalizados e bits de guarda. O sticky bit fica a 0 se os bits não incluídos no produto forem todos zero e a 1 se acontecer o contrário: este bit seria usado no bloco arredondamento se esse tivesse sido implementado.

Bloco norm_mul_v2:

O bloco da normalização é sem dúvida o bloco mais difícil de implementar pois, considerando todos os casos possíveis que os operandos denormal acrescentam, há uma preocupação extrema em simplificar o mais possível a rotina, evitando ter de definir linhas de código adicionais para casos que o ciclo *for* não contempla (à base de *if*s).

Tal como foi feito na subtracção, o ciclo *for* vai percorrendo todos os bits da entrada passados pelo bloco da multiplicação, procurando o primeiro 1 e caso o expoente e o hidden bit sejam diferentes de 0, e vai shiftando o número para a esquerda, caso o expoente seja diferente de 1 (e 0). A explicação de o operando não shiftar para a esquerda quando auxexp é 1 reside no facto de que quando aux_exp atinge 1, o hidden bit fica a 0, perdendo o operando com isso uma casa binária (logo não é necessário shiftá-lo).

Ao fazer-se o shift pode acontecer que o hidden bit fique na posição mais significativa e o expoente a zero. Neste caso, incrementa-se o expoente (coloca-se a 1) – "*if(!auxexp & aux_nor[NBM+NBG+1]) auxexp=1;*".

Finalmente são vistos os casos de overflow e underflow, em que o primeiro acontece quando o bit mais significativo do expoente é 1 e o resto do expoente diferente de 0 (auxexp=1x…x, com x…x $\neq$ 0), e o segundo quando o expoente é 0 e os bits da mantissa "lançados" para o bloco seguinte (hidden bit, bits de guarda e bits normalizados da mantissa) são zero.

Secção 1.04 Divisão

A divisão, ao contrário da multiplicação, soma e subtracção, não considera os operandos denormal. Isto deve-se não só à não exigência desta característica pelo professor, mas também porque tornaria muito complexo o já tão complexo circuito da divisão, nomeadamente no que respeita à normalização, que aqui teria de shiftar para a direita ou esquerda o operando final, com todas os ajustes de expoente e mantissa que isso implicaria.

O expoente final caso não houvesse carry na divisão é a subtracção dos expoentes dos operandos mais o BIAS.

O bloco que punha hidden bit foi agora substituído por duas linhas de código, já que agora é só colocar um 1 no bit mais significativo dos operandos com hidden bit, ficando o resto dos bits com os bits dos operandos de entrada.

É depois realizada a operação de divisão propriamente dita (bloco div para divisão combinacional ou bloco div_seq para divisão sequencial). Para se utilizar a divisão combinacional basta comentar o bloco div_seq, enquanto que para se utilizar a divisão sequencial comenta-se o bloco div.

É depois realizada a normalização (norm_div).

Na divisão, como não há cálculo do sticky bit, então arred_nor tem apenas NBM+NBG+1 bits. No entanto, como o bloco de arredondamento é comum a todas as operações, é necessário mandar um número de bits igual, logo manda-se o sticky bit a zero e os outros bits na posição respectiva.

As flags overflow e underflow são determinadas no bloco da normalização e passadas para fora da divisão em ovfd e udfd.

Bloco div:

Este bloco (divisão) tenta reproduzir uma divisão "à primária", baseado no divisor shift-subtract. O divisor unsigned das folhas de vírgula flutuante exigia o dobro do número de ciclos (aumentando o tempo de execução sensivelmente para o dobro).

O ciclo repete-se NBM+NBG+1 vezes, ou seja, quantas vezes quanto o grau de precisão que queremos; claro que o número de bits das variáveis A e Q também deverá acompanhar este aumento de precisão.

O algoritmo da divisão é simples: se A for maior que D então a posição menos significativa de Q fica com 1 e A é decrementado de D; caso não, a posição menos significativa de Q fica com 0 e A mantém-se (ao passar dos ciclos A mantém-se até que, sujeito aos shifts, fique superior a D, altera em que D lhe é decrementado). Finalmente shifta-se a concatenação de A e Q para a esquerda. Na saída não queremos o bit menos significativo de Q pois é sempre 0, já que Q é sempre sujeito na instrução anterior a um shift para a esquerda.

Bloco div_seq

Este é o bloco da divisão sequencial.

Utiliza o método shift-subtract tal como o bloco div. As diferenças residem na adaptação para o modo sequencial. Primeiramente define-se 2 state, um que vai verificando se o sinal start foi accionado e que se tal acontece, coloca em A o dividendo, em D o divisor e em Q coloca o 0. Quando o start é activado, passa para o state=1, que corresponde a um ciclo em que cada iteração é activada por uma transição positiva de clock (posedge clock) até se realizarem NBM+NBG+1 iterações. Nestas iterações vão sendo realizadas as operações já descritas pormenorizadamente na descrição do bloco div.

Bloco norm_div

Este é o bloco da normalização da divisão. Visto não contar com operandos denormal, a sua síntese não é muito complexa. Tem de se verificar se o bit mais significativo é 0, e caso seja, shifta-se a mantissa um bit para a esquerda e decrementa-se o expoente de uma unidade.

Definiu-se depois as flags overflow (ovf) e underflow (udf). Overflow acontece quando o expoente de entrada é 10…0 (1º bit a 1 e os outros a 0) ou o expoente de saída é 011…1(1º bit a 0 e os outros a 1). Underflow ocorre quando o expoente de entrada é 0 ou 1…1x (todos os bits a 1 excepto o menos significativo que pode ser um qualquer) ou quando o expoente de saída é 0.

Secção 1.05 Arredondamento

O bloco de arredondamento (arred) trunca simplesmente a mantissa do resultado final para a dimensão que ela terá de apresentar na saída (NBM bits); o expoente é simplesmente reencaminhado para a saída.

MEDIÇÕES

Os valores seguintes foram obtidos com a FPGA xcv300e-6fg456.

Todos os tempos estão em nanosegundos e o formato de 8 bits foi definido com 4 bits de mantissa e 3 para o expoente, o de 16 bits: 9 de mantissa e 6 de expoente e o formato simples (32 bits), com os definidos na norma, de 23 para a mantissa e 8 bits para o expoente.

Para todos os módulos sequenciais só foi gerado um gated clock.

Secção 1.01 Valores especiais

Bloco principal (val_exp_geral):

	Optimização Área				Optimização Velocidade			
Bits	Tempo de delay	Slices	4 Input LUTs	bonded IOBs	Tempo de delay	Slices	4 Input LUTs	bonded IOBs
8	22,271	28	50	29	17,449	39	76	29
16	25,047	32	57	53	18,637	46	89	53
32	29,011	41	72	101	20,203	63	124	101

Bloco val_exp:

	Optimização Área				Optimização Velocidade			
Bits	Tempo de delay	Slices	4 Input LUTs	bonded IOBs	Tempo de delay	Slices	4 Input LUTs	bonded IOBs
8	9,925	8	14	26	9,695	7	14	26
16	14,089	13	22	42	11,313	14	26	42
32	18,253	20	34	74	20,203	23	45	74

Bloco val_exp3:

	Optimização Área				Optimização Velocidade			
Bits	Tempo de delay	Slices	4 Input LUTs	bonded IOBs	Tempo de delay	Slices	4 Input LUTs	bonded IOBs
8	8,707	2	3	13	7,487	2	3	13
16	9,237	2	3	21	7,487	2	3	21
32	9,437	2	3	37	7,487	2	3	37

Bloco val_exp2_v2:

O comportamento do bloco val_exp2_v2 não depende do número de bits dos operandos, pois as suas entradas não dependem directamente destes. Assim, temos os mesmos tempos e componentes para qualquer número de bits dos operandos.

Optimização Área				Optimização Velocidade			
Tempo de delay	Slices	4 Input LUTs	bonded IOBs	Tempo de delay	Slices	4 Input LUTs	bonded IOBs
15,689	19	34	20	13,381	25	47	20

Secção 1.02 Multiplicação

Multiplicação combinacional (multiplicação_geral):

	Optimização Área				Optimização Velocidade			
Bits	Tempo de delay	Slices	4 Input LUTs	bonded IOBs	Tempo de relay	Slices	4 Input LUTs	bonded IOBs
8	30,871	148	258	30	27,728	174	327	30
16	110,088	324	578	54	105,906	579	1114	54
32	274,010	1175	2127	102	204,609	1302	2482	102

Multiplicação sequencial (multiplicação_geral):

	Optimização Área						
Bits	Delay combinacional	Período mínimo unitário	Período mínimo total	Slices	Slice Flip Flops	4 Input LUTs	bonded IOBs
8	25,383	9,848	39,392	160	16	284	32
16	107,678	12,278	110,502	299	27	531	56
32	266,066	12,828	295,044	942	56	1674	104

	Optimização Velocidade						
Bits	Delay combinacional	Período mínimo unitário	Período mínimo total	Slices	Slice Flip Flops	4 Input LUTs	bonded IOBs
8	22,552	10,528	42,112	201	32	387	32
16	102,586	11,928	107,352	670	28	1288	56
32	197,143	14,216	326,968	1109	57	2133	104

Secção 1.03 Divisão

Divisão combinacional (divisao_geral):

	Optimização Área				Optimização Velocidade			
Bits	Tempo de delay	Slices	4 Input LUTs	bonded IOBs	Tempo de delay	Slices	4 Input LUTs	bonded IOBs
8	46,047	88	112	27	40,461	90	117	27
16	80,155	314	396	51	74,066	320	413	51
32	210,174	1650	1919	99	200,034	1639	1986	99

Divisão sequencial (divisão_geral)

Optimização Área							
Bits	Delay combinacional	Período mínimo unitário	Período mínimo total	Slices	Slice Flip Flops	4 Input LUTs	bonded IOBs
8	12,162	12,574	50,296	52	51	93	30
16	22,028	12,574	113,166	86	68	151	54
32	22,032	13,049	300,127	138	110	246	102

Optimização Velocidade							
Bits	Delay combinacional	Período mínimo unitário	Período mínimo total	Slices	Slice Flip Flops	4 Input LUTs	bonded IOBs
8	12,446	10,911	43,644	60	52	95	30
16	19,322	11,186	100,674	88	68	160	54
32	20,672	11,686	268,778	146	112	271	102

Secção 1.04 Soma e Subtracção

Soma e subtracção geral (somasub_geral):

	Optimização Área				Optimização Velocidade			
Bits	Tempo de delay	Slices	4 Input LUTs	bonded IOBs	Tempo de delay	Slices	4 Input LUTs	bonded IOBs
8	40,971	104	184	32	34,490	149	278	32
16	147,101	313	539	56	118,667	527	990	56
32	287,855	1029	1781	104	219,202	1155	2169	104

Secção 1.05 Circuito Geral

VFL geral combinacional (VFLunit):

Circuito inteiramente combinacional:

	Optimização Área				Optimização Velocidade			
Bits	Tempo de delay	Slices	4 Input LUTs	bonded IOBs	Tempo de delay	Slices	4 Input LUTs	bonded IOBs
8	44,827	330	545	31	38,169	401	709	31
16	152,131	975	1559	55	118,529	1347	2356	55
32	288,768	3818	5782	103	229,690	4562	7570	103

A laranja estão os casos em que se excedeu o número de unidades máximo da FPGA.

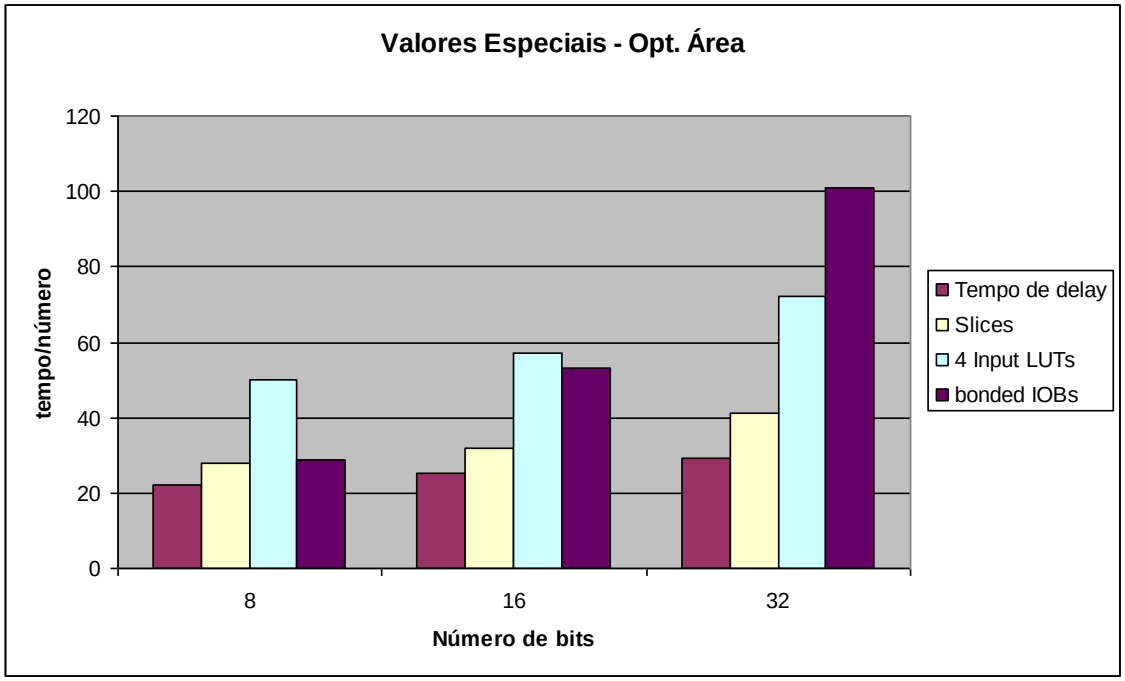
VFL geral sequencial (VFLunit):

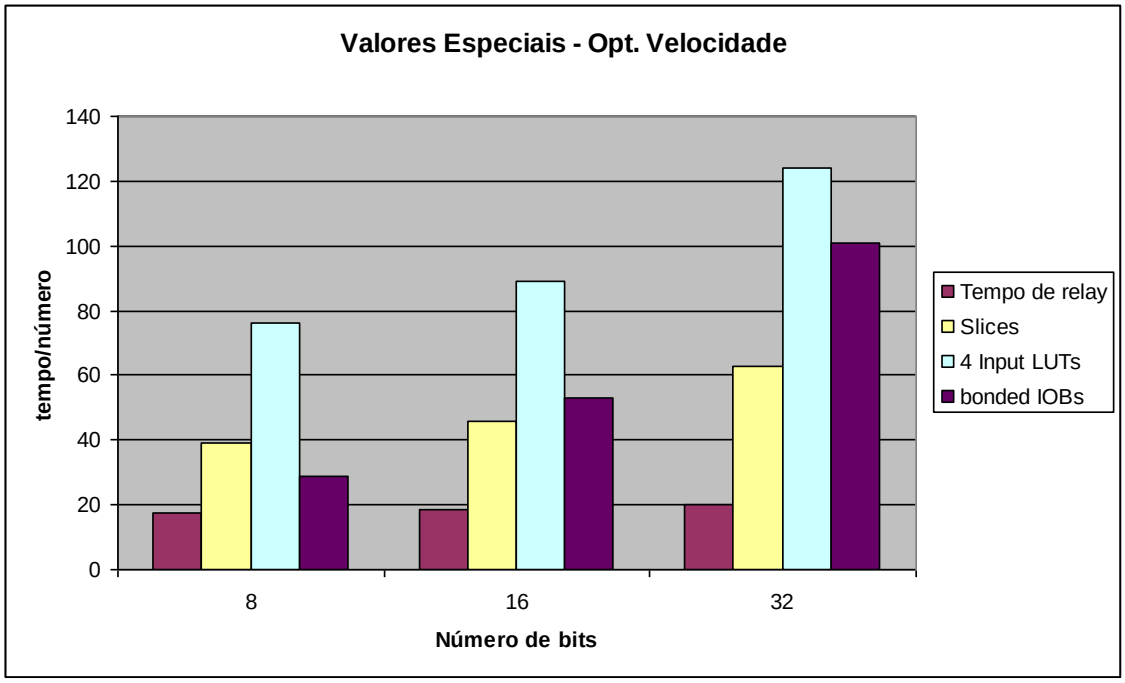
Operações de multiplicação e divisão sequenciais e o resto do circuito combinacional:

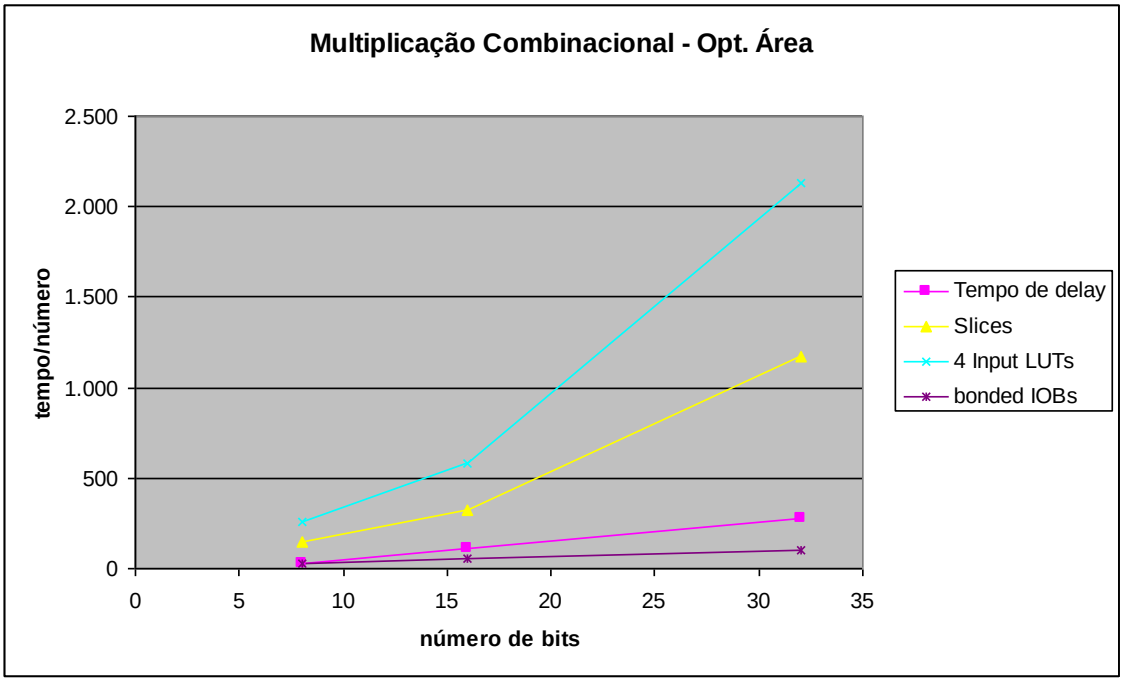
Optimização Área							
Bits	Delay combinacional	Período mínimo unitário	Período mínimo total	Slices	Slice Flip Flops	4 Input LUTs	bonded IOBs
8	44,777	12,574	50,296	326	68	577	33
16	151,182	12,574	113,166	763	94	1329	57
32	286,268	13,049	300,127	2174	165	3808	105

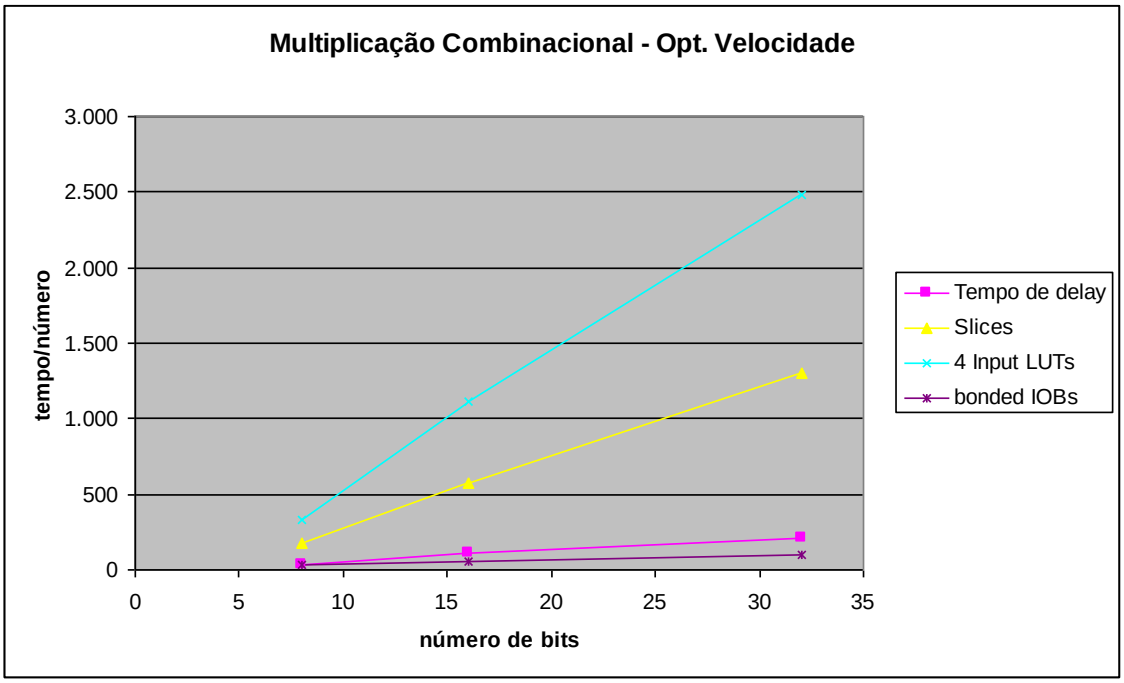
Optimização Velocidade							
Bits	Delay combinacional	Período mínimo unitário	Período mínimo total	Slices	Slice Flip Flops	4 Input LUTs	bonded IOBs
8	35,582	11,186	44,744	326	68	577	33
16	118,357	11,928	107,352	1255	95	2374	57
32	218,945	14,216	326,968	2069	168	3909	105

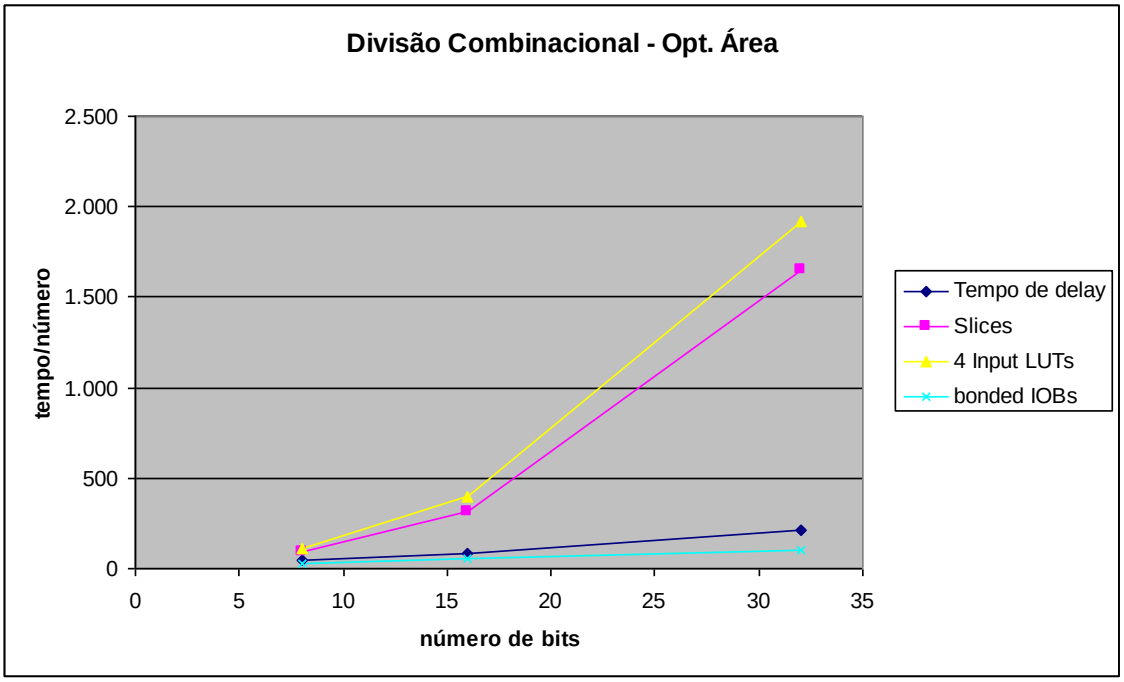
GRÁFICOS

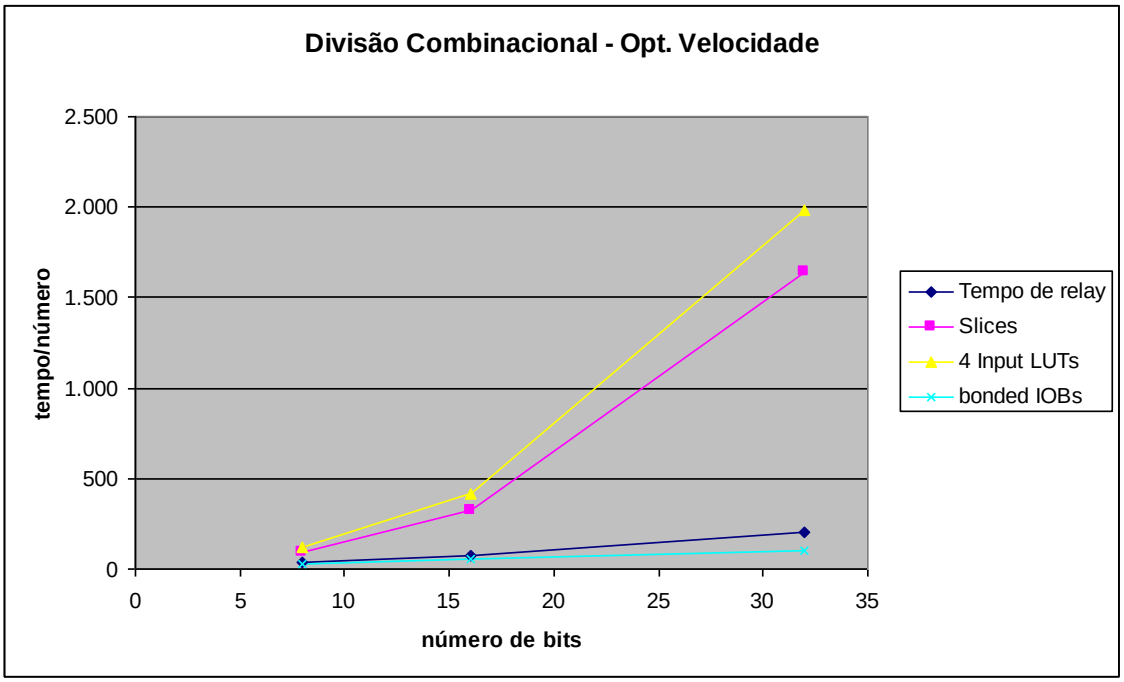

Valores Especiais - Opt. Área
120
100
80
60
40
20
0
tempo/número
8
16
32
Número de bits
Tempo de delay
Slices
4 Input LUTs
bonded IOBs

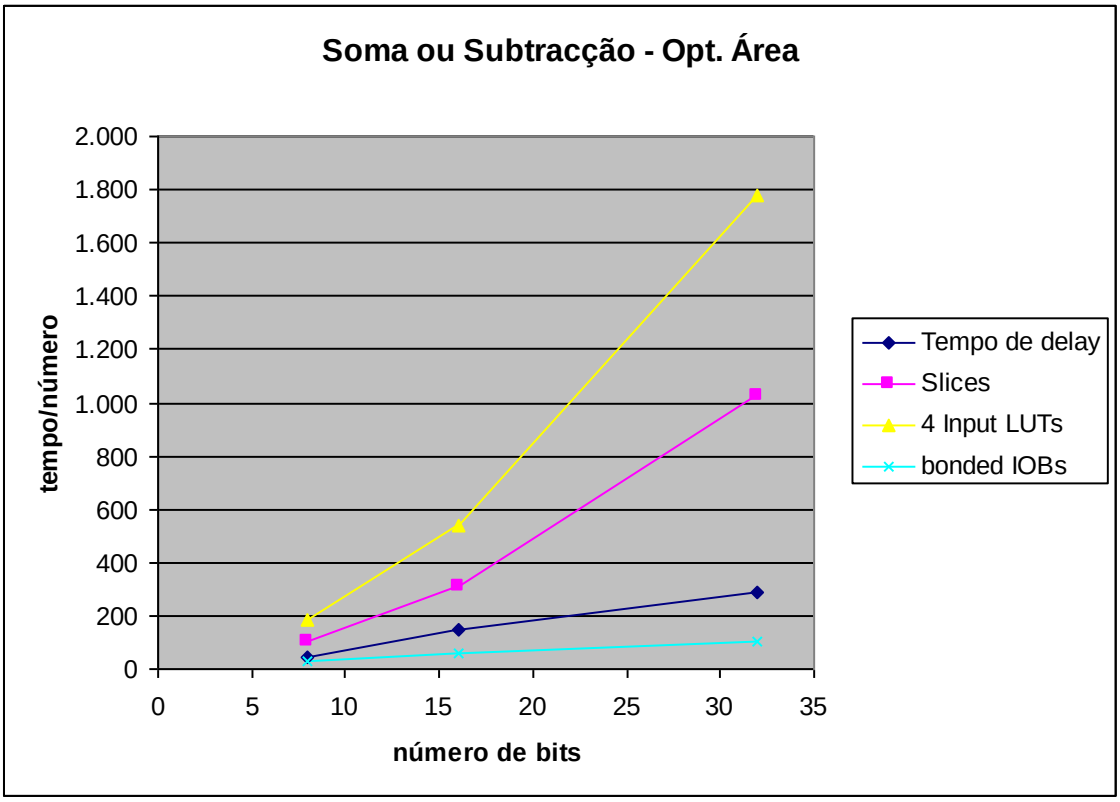

Valores Especiais - Opt. Velocidade
140
120
100
80
60
40
20
0
tempo/número
8
16
32
Número de bits
Tempo de relay
Slices
4 Input LUTs
bonded IOBs

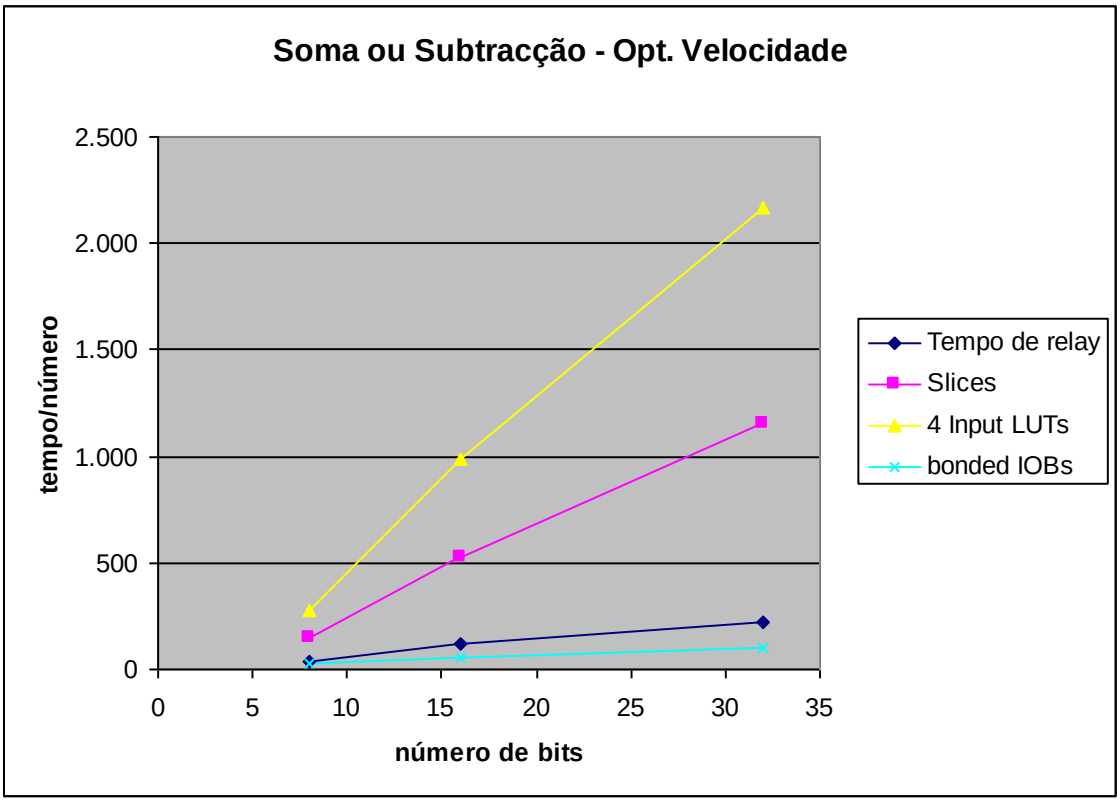

Multiplicação Combinacional - Opt. Área
tempo/número
2.500
2.000
1.500
1.000
500
0
0
5
10
15
20
25
30
35
número de bits
Tempo de delay
Slices
4 Input LUTs
bonded IOBs

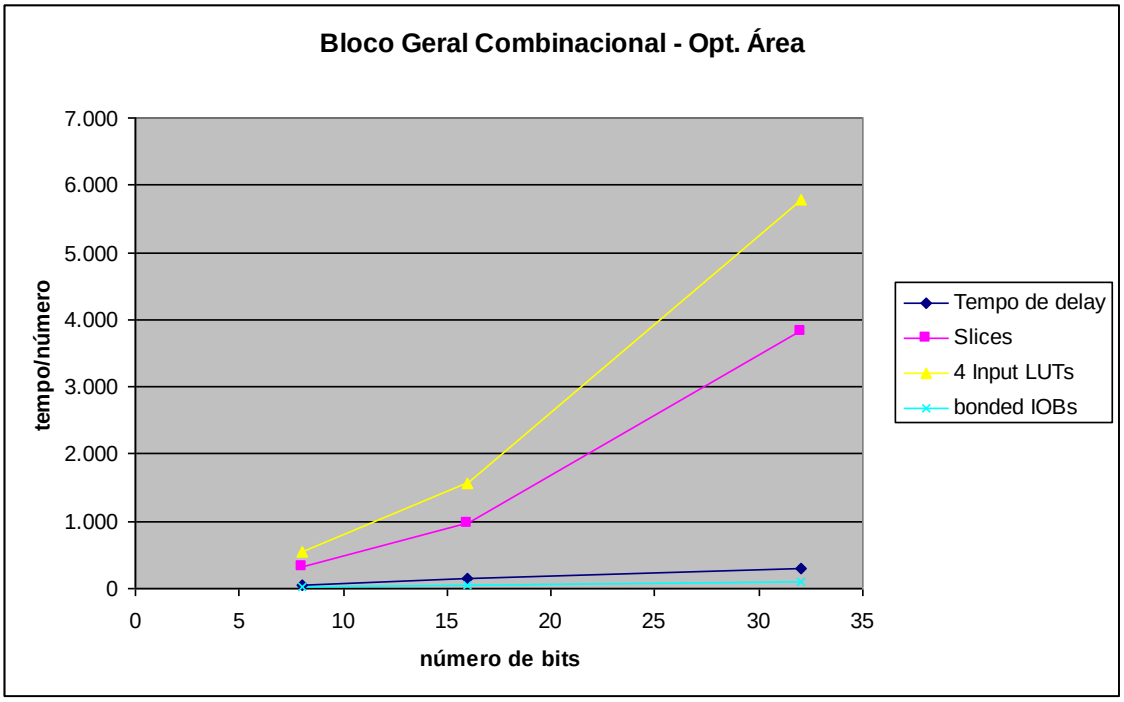

Multiplicação Combinacional - Opt. Velocidade
tempo/número
3.000
2.500
2.000
1.500
1.000
500
0
0
5
10
15
20
25
30
35
número de bits
Tempo de delay
Slices
4 Input LUTs
bonded IOBs


Divisão Combinacional - Opt. Área
2.500
2.000
1.500
1.000
500
0
tempo/número
0
5
10
15
20
25
30
35
número de bits
Tempo de delay
Slices
4 Input LUTs
bonded IOBs

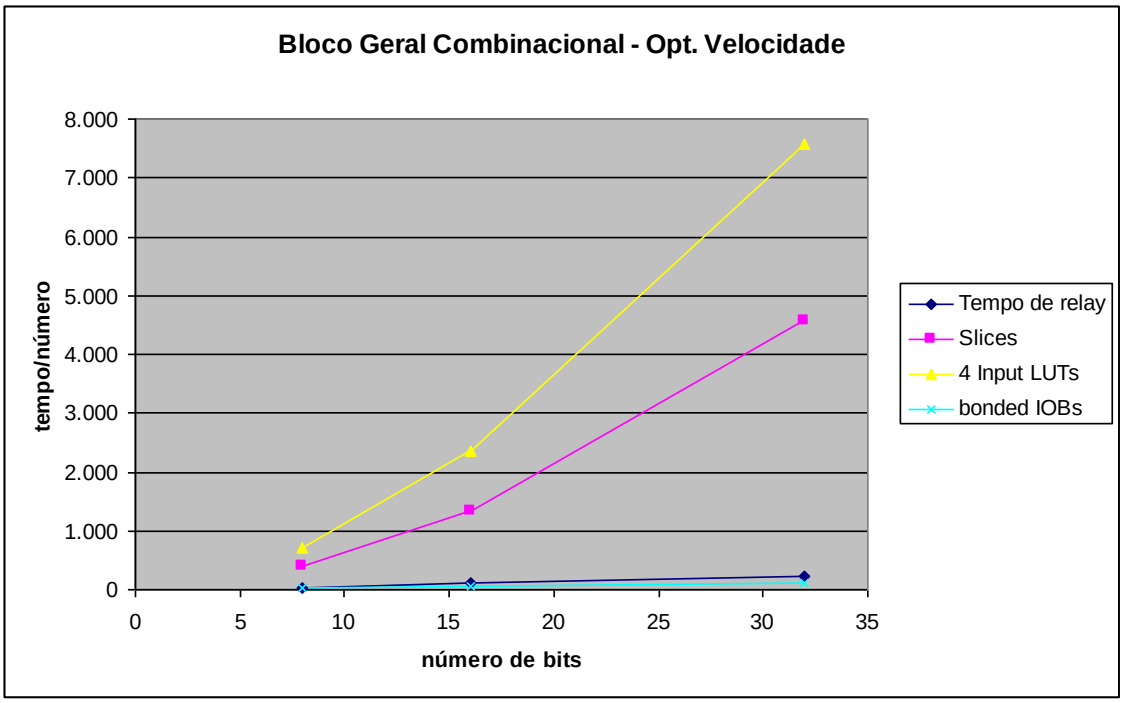

Divisão Combinacional - Opt. Velocidade
2.500
2.000
1.500
1.000
500
0
tempo/número
0
5
10
15
20
25
30
35
número de bits
Tempo de relay
Slices
4 Input LUTs
bonded IOBs


Soma ou Subtracção - Opt. Área
2.000
1.800
1.600
1.400
1.200
1.000
800
600
400
200
0
tempo/número
0
5
10
15
20
25
30
35
número de bits
Tempo de delay
Slices
4 Input LUTs
bonded IOBs

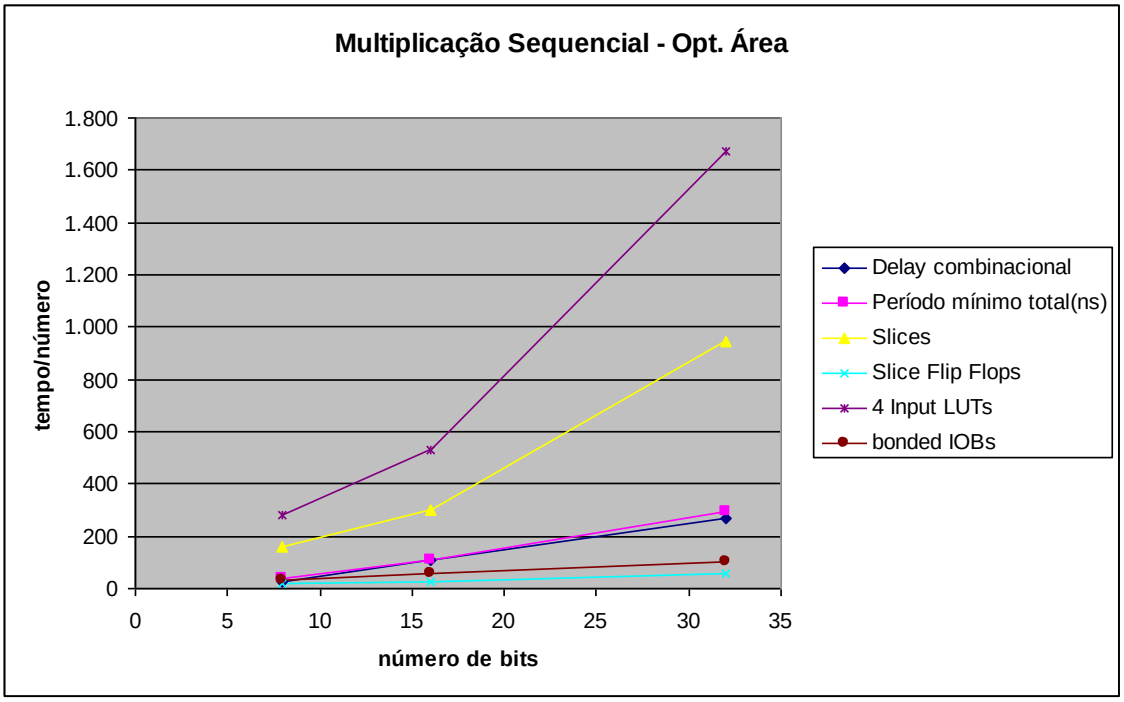

Soma ou Subtracção - Opt. Velocidade
2.500
2.000
1.500
1.000
500
0
tempo/número
0
5
10
15
20
25
30
35
número de bits
Tempo de relay
Slices
4 Input LUTs
bonded IOBs

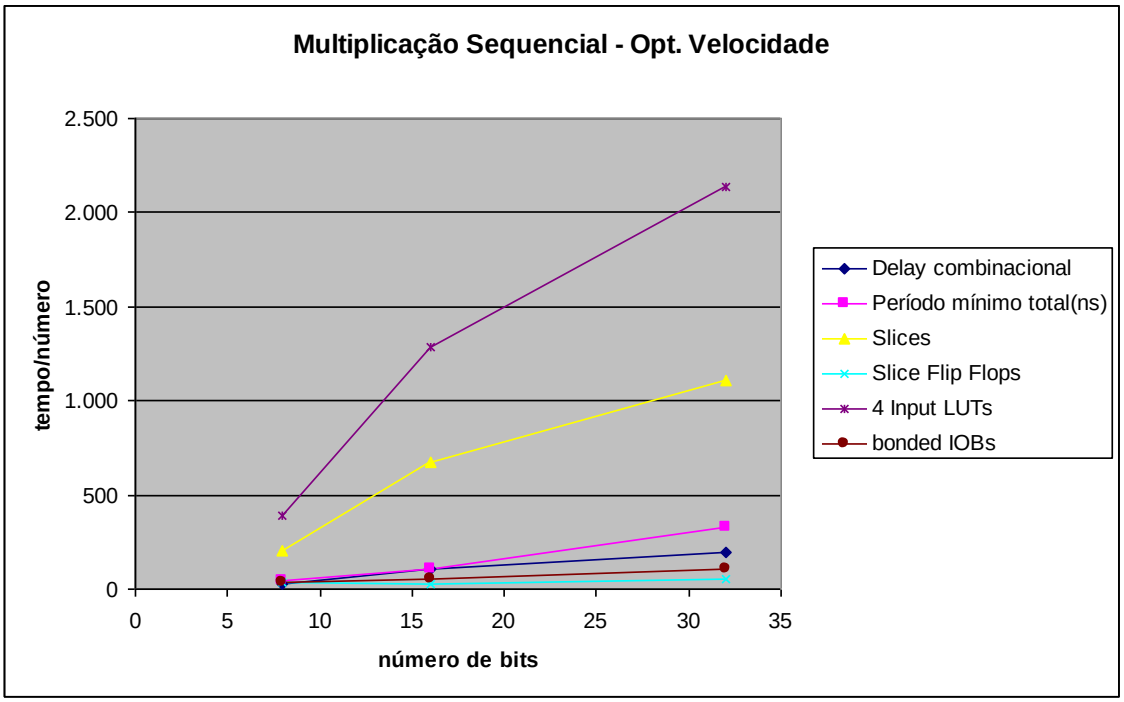


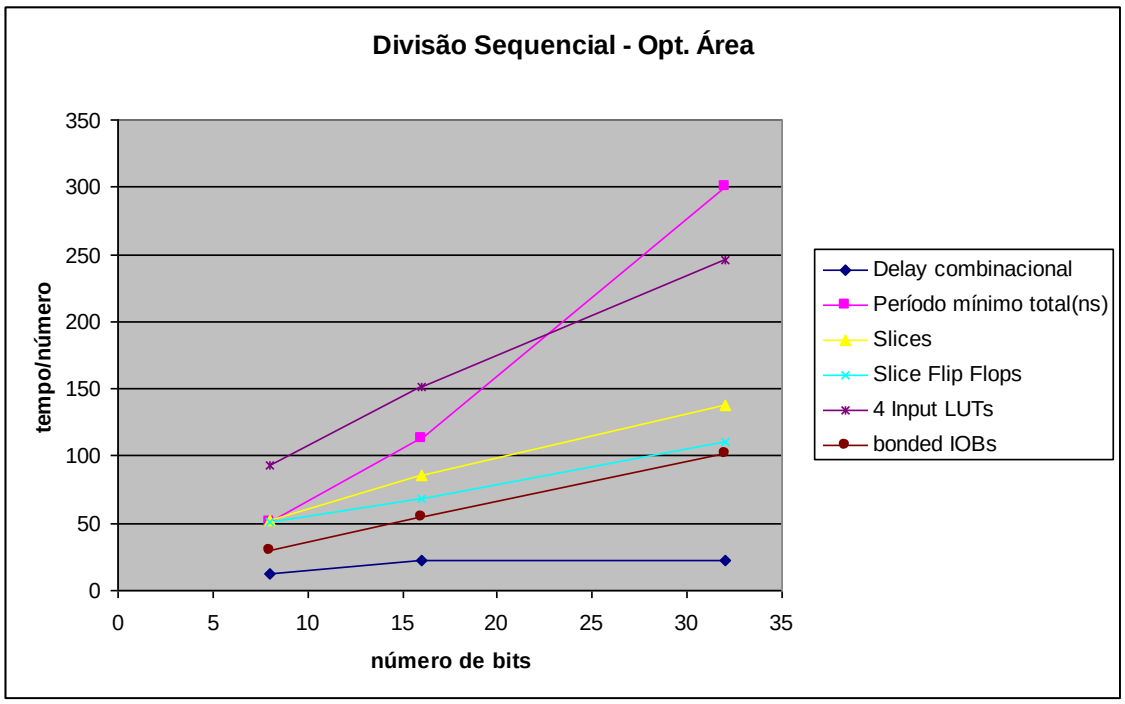
Os slices para 32 bits excederam o tamanho da FPGA.

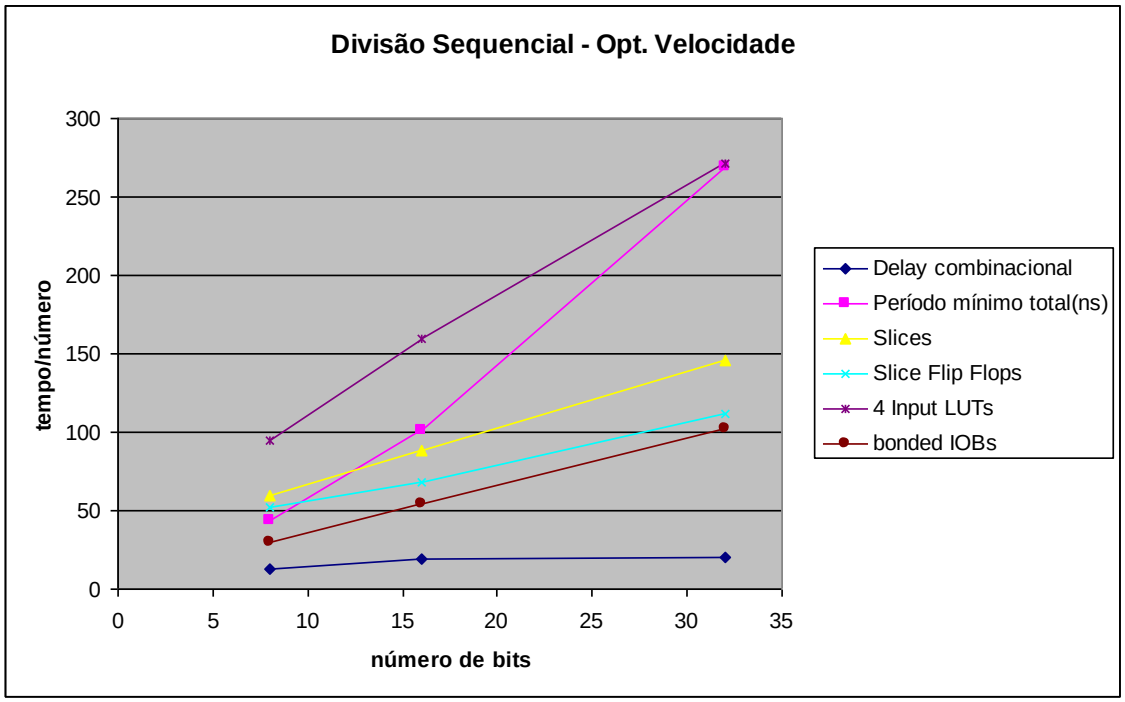


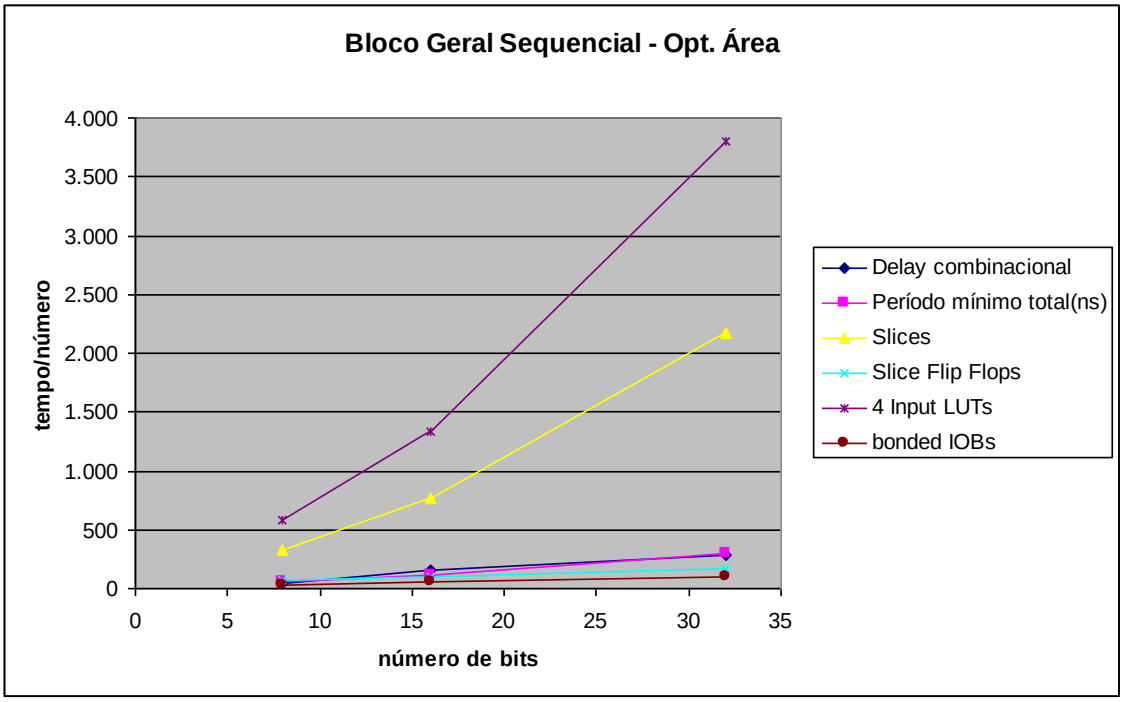
Os slices e LUTs para 32 bits excederam o tamanho da FPGA.

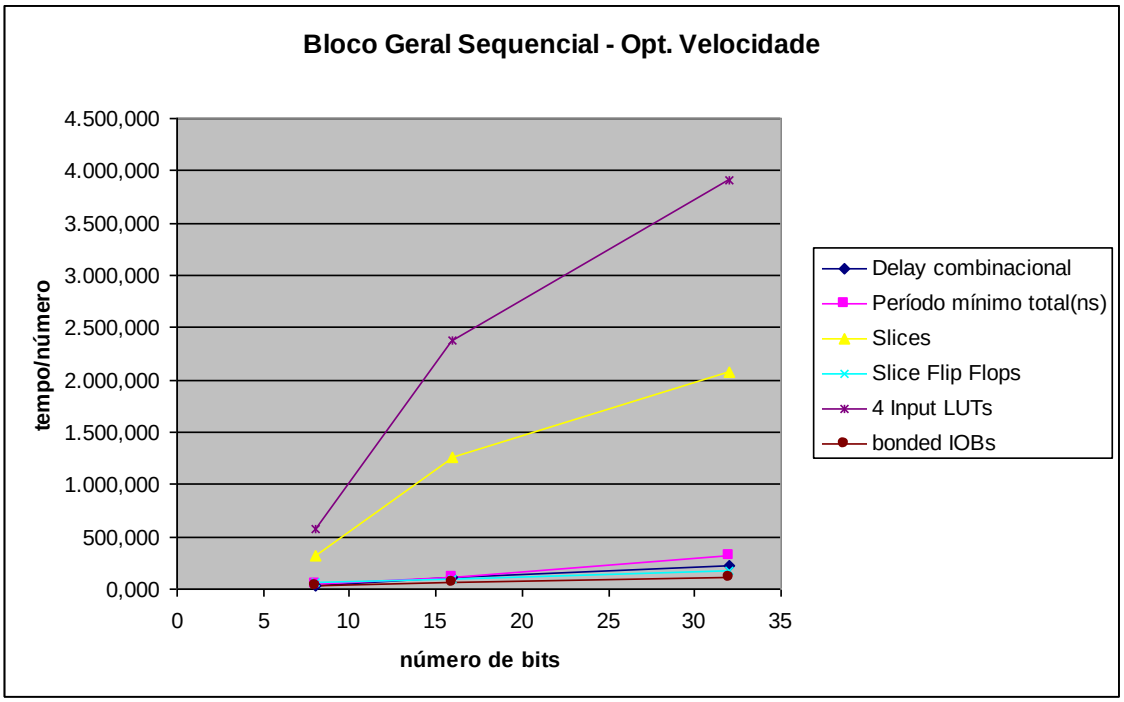

Multiplicação Sequencial - Opt. Área
tempo/número
1.800
1.600
1.400
1.200
1.000
800
600
400
200
0
0
5
10
15
20
25
30
35
número de bits
Delay combinacional
Período mínimo total(ns)
Slices
Slice Flip Flops
4 Input LUTs
bonded IOBs


Multiplicação Sequencial - Opt. Velocidade
tempo/número
2.500
2.000
1.500
1.000
500
0
0
5
10
15
20
25
30
35
número de bits
Delay combinacional
Período mínimo total(ns)
Slices
Slice Flip Flops
4 Input LUTs
bonded IOBs


Divisão Sequencial - Opt. Área
tempo/número
350
300
250
200
150
100
50
0
0
5
10
15
20
25
30
35
número de bits
Delay combinacional
Período mínimo total(ns)
Slices
Slice Flip Flops
4 Input LUTs
bonded IOBs


Divisão Sequencial - Opt. Velocidade
tempo/número
300
250
200
150
100
50
0
0
5
10
15
20
25
30
35
número de bits
Delay combinacional
Período mínimo total(ns)
Slices
Slice Flip Flops
4 Input LUTs
bonded IOBs


Bloco Geral Sequencial - Opt. Área
tempo/número
4.000
3.500
3.000
2.500
2.000
1.500
1.000
500
0
0
5
10
15
20
25
30
35
número de bits
Delay combinacional
Período mínimo total(ns)
Slices
Slice Flip Flops
4 Input LUTs
bonded IOBs


Bloco Geral Sequencial - Opt. Velocidade
tempo/número
4.500,000
4.000,000
3.500,000
3.000,000
2.500,000
2.000,000
1.500,000
1.000,000
500,000
0,000
0
5
10
15
20
25
30
35
número de bits
Delay combinacional
Período mínimo total(ns)
Slices
Slice Flip Flops
4 Input LUTs
bonded IOBs

ANÁLISE DE RESULTADOS

O bloco de **valores especiais** é o bloco que passa o resultado para a saída mais rapidamente. Devido à maneira como foi implementado (quase independente do número de bits dos operandos de entrada ou saída), os valores de delay para diferentes números de bits dos operandos são bastante próximos.

Em relação à **multiplicação**, comparando o tempo de atraso da versão combinacional com o período mínimo total por nós calculado (com base na multiplicação do período mínimo unitário pelo número de bits da mantissa), conclui-se que a multiplicação combinacional é sempre mais rápida a executar que a sequencial. Em relação à área ocupada, a solução combinacional mostra-se vantajosa para 8 bits, equilibrada para 16 bits e ligeiramente pior para 32 bits quando comparada com a solução sequencial. Conclui-se assim, que para formatos de vírgula flutuante de tamanho inferior ao simples, é em geral preferível, em termos de área e velocidade, a nossa solução combinacional à sequencial. Outras soluções de cálculo de multiplicação sequencial poderiam ter sido implementadas, pelo que se teria de avaliar se essas soluções seriam mais optimizadas comparativamente com a combinacional.

A **divisão**, tal como a multiplicação, é sempre mais rápida a executar na solução combinacional do que na sequencial. Já em termos de área ocupada, embora para 8 e 16 bits ainda se possa considerar aceitável, de modo a implementar um divisor de 32 bits combinacional passa-se a ocupar 5 a 6 vezes mais área que a solução sequencial para o mesmo formato de vírgula flutuante. O facto de o circuito integral totalmente combinacional a 32 bits execeder o tamanho da FPGA deve-se essencialmente a este operador e ao módulo somador/subtractor.

Comparando os tempos para 8 bits, vemos que, tal como seria de esperar, a divisão é a mais lenta das operações. Mas para 16 e 32 bits esta demonstrou-se a mais rápida de todas; isto deve-se essencialmente ao facto de ser a única em que não foi implementado o uso de números denormalizados, demonstrando-se assim a grande diferença de complexidade entre o uso do formato pela norma ou somente supondo o hidden bit sempre a 1.

O módulo geral da **soma/subtracção** comparativamente com o da multiplicação demonstra ser mais lento na execução de uma operação, qualquer que seja o número de bits usado. Isto deve-se ao uso de mais submódulos para a preparação da operação em si que a multiplicação. Tal como a divisão, para 32bits, esta solução para a soma/subtracção ocupa bastante área.

ANEXOS

A.

Aqui estão apresentadas medições de alguns submódulos das operações e arredondamento para 8 bits; NBM=4 e NBE=3 (Usando a FPGA XCV400, HQ240-6 – pedida no guião).

Secção 1.01 Soma

Módulo "som_ou_sub":

Optimização Área				Optimização Velocidade			
Tempo de delay	Slices	4 Input LUTs	bonded IOBs	Tempo de delay	Slices	4 Input LUTs	bonded IOBs
7,937	1	2	7	7,117	1	2	7

Ao retirar-se os registos e passando-se a usar *assign* no código verilog, os resultados foram idênticos para os dois casos (tanto para optimização de tempo, área) em termos de tempo e área ocupada.

Módulo "max":

Optimização Área				Optimização Velocidade			
Tempo de delay	Slices	4 Input LUTs	bonded IOBs	Tempo de delay	Slices	4 Input LUTs	bonded IOBs
16,691	14	25	32	13,797	14	25	32

Módulo "põe_hidden": (para virtexE XCV300e 6FG456)

Optimização Área				Optimização Velocidade			
Tempo de delay	Slices	4 Input LUTs	bonded IOBs	Tempo de delay	Slices	4 Input LUTs	bonded IOBs
7,707	1	2	24	6,887	1	2	24

Bloco "alinhamento":

Optimização Área				Optimização Velocidade			
Tempo de delay	Slices	4 Input LUTs	bonded IOBs	Tempo de delay	Slices	4 Input LUTs	bonded IOBs
12,530	7	13	14	9,906	6	12	14

Bloco "Somasub":

Optimização Área				Optimização Velocidade			
Tempo de delay	Slices	4 Input LUTs	bonded IOBs	Tempo de delay	Slices	4 Input LUTs	bonded IOBs
10,406	4	8	21	10,406	4	8	21

Bloco "norm_v2":

Optimização Área				Optimização Velocidade			
Tempo de delay	Slices	4 Input LUTs	bonded IOBs	Tempo de delay	Slices	4 Input LUTs	bonded IOBs
20,748	70	125	23	17,340	78	149	23

Secção 1.02 Arredondamento

Bloco de arredondamento ("arred"):

Optimização Área				Optimização Velocidade			
Tempo de delay	Slices	4 Input LUTs	bonded IOBs	Tempo de delay	Slices	4 Input LUTs	bonded IOBs
6,590	0	0	14	6,590	0	0	14

Secção 1.03 Multiplicação

Bloco principal ("multiplicacao"):

Optimização Área				Optimização Velocidade			
Tempo de delay	Slices	4 Input LUTs	bonded IOBs	Tempo de delay	Slices	4 Input LUTs	bonded IOBs
15,694	16	24	20	15,694	16	24	20

Bloco "põe_hidden":

Optimização Área				Optimização Velocidade			
Tempo de delay	Slices	4 Input LUTs	bonded IOBs	Tempo de delay	Slices	4 Input LUTs	bonded IOBs
8,198	1	2	24	7,263	1	2	24

Bloco "norm_mul":

Optimização Área				Optimização Velocidade			
Tempo de delay	Slices	4 Input LUTs	bonded IOBs	Tempo de delay	Slices	4 Input LUTs	bonded IOBs
17,543	30	52	24	13,155	32	60	24